

AI DevOps Incident Post-Mortem Report

Severity: HIGH

Service: web-server

****Post-Mortem Report: High Severity Web-Server Incident****

1. Executive Summary

On [Date], our web-server experienced a high-severity incident resulting in significant performance degradation and intermittent downtime. The root cause was identified as an increased CPU load following a recent deployment. This report outlines the incident's timeline, root cause, impact, resolution, and provides recommendations for preventing similar incidents in the future.

2. Timeline

- ****[Date] 14:00****: Recent deployment of new code changes to the web-server.
- ****[Date] 14:15****: Initial reports of web-server performance issues, including slow response times and errors.
- ****[Date] 14:30****: Monitoring systems alerted the DevOps team to a significant increase in CPU load on the web-server.
- ****[Date] 15:00****: The incident was escalated due to the requirement for manual intervention, as automated processes were insufficient to resolve the issue.
- ****[Date] 16:00****: DevOps team began manual intervention to mitigate the issue, including rolling back recent code changes and optimizing server configurations.
- ****[Date] 17:30****: Web-server performance was stabilized, and services were restored to normal operation.

3. Root Cause

The root cause of the incident was an increased CPU load on the web-server following a recent deployment. Analysis revealed that the new code changes introduced inefficient database queries and inadequate caching mechanisms, leading to a substantial increase in computational demands on the server. This increase in CPU load resulted in performance degradation, slow response times, and intermittent service unavailability.

4. Impact

The incident had a high severity impact on our services, resulting in:

- ****Service Downtime****: The web-server experienced intermittent downtime for approximately 2 hours, affecting user access to our platform.
- ****Performance Degradation****: Even when the service was available, users experienced slow response times and errors, leading to a significant decrease in user satisfaction.
- ****Business Impact****: The downtime and performance issues resulted in a loss of business opportunities and revenue during the affected period.

5. Resolution

Resolution of the incident involved:

- **Rollback of Recent Code Changes**: The DevOps team rolled back the recent deployment to a previous stable version, immediately alleviating some of the CPU load.
- **Optimization of Server Configurations**: Temporary adjustments were made to server configurations to better handle the load until a permanent fix could be implemented.
- **Implementation of Emergency Patches**: Emergency patches were applied to address the inefficient database queries and caching issues identified in the new code.

6. Prevention Recommendations

To prevent similar incidents in the future, we recommend:

- **Thorough Code Review**: Implementing a more rigorous code review process to catch potential performance issues before deployment.
- **Automated Performance Testing**: Integrating automated performance testing into our CI/CD pipeline to identify and flag inefficient code changes before they reach production.
- **Enhanced Monitoring and Alerting**: Improving our monitoring and alerting systems to detect early signs of performance degradation, allowing for quicker intervention.
- **Regular Load Testing**: Conducting regular load testing to ensure our infrastructure can handle expected and unexpected spikes in traffic and computational demands.
- **Continuous Deployment of Fixes**: Continuously deploying fixes and optimizations to ensure that our web-server remains performant and reliable.